

class 6: R functions

Giselle Garcia A17995734

Table of contents

Background	1
Afirst fuction	1
A generate_dna() fuction	3
Write a generate_protein() fuction	6
Generate random protein sequences of length 6 to 13	6
BLASTp search against nr — are your peptides “unique in nature”?	7
connecting your finding to Immunology	7

Background

All function in R have at least 3 things:

- a *name* (we pick that and use it to call the fuction) -Input *arguments* (one or more comma operated inputs that got inside the brackets when we call the function) -the *body* (the lines of the R code that doe the week of the function)

Afirst fuction

here we will create a function to add some numbers. Let's call it `add()`

Input argument can be either “**required**” or “**optional**”. The later have fall-back default values that will be used if the user does not specify them.

```
add<- function(x,y=0) {  
  x+y  
}
```

Can we use our new function:

```
add(10,100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 10
```

Q1b: For your second version, adapt your first function so it can take a single input vector or two inputs as before. For example, add(4, 7) and add (c(4,7,10)).

```
add<- function(x,y=0) {  
  sum(x+y)  
}  
add(4,7)
```

```
[1] 11
```

```
add(c(4,7,10))
```

```
[1] 21
```

Q1c. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, add(1, 2, 3, -4)

```
add<- function(x,y=0, z=0) {  
  sum(x,y,z)  
}
```

```
add(10,100)
```

```
[1] 110
```

```
ans <- add (1,2,3)  
ans
```

```
[1] 6
```

#this function adds any number of inputs provided

```
add<- function(...) {
  sum(...)
}
```

```
add(1,2,3,-4)
```

```
[1] 2
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the **return()** function call.

```
add<- function(x,y=0, z=0) {
  return(sum(x,y,z))
  cat("Is it break time yet")
}
```

A generate_dna() function

A useful function here is the “Base R” **sample()** function:

```
sample(1:5, size=3)
```

```
[1] 5 1 3
```

We can use this to make a random nucleotide sequence if we draw “A”, “C”, “G” and “T”...

```
sample(1:5, size=60, replace= TRUE)
```

```
[1] 5 5 4 2 5 4 1 5 2 5 3 2 2 4 2 1 1 5 3 2 5 4 2 2 1 5 3 4 2 1 3 5 3 2 3 2 3 5
[39] 2 1 4 2 3 2 3 1 5 5 1 5 4 3 1 2 5 3 3 3 3 3
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G” and “T”...

```
sample(x=c("A", "C", "G","T"), size= 10, replace= TRUE)
```

```
[1] "A" "C" "A" "A" "A" "C" "T" "G" "C" "T"
```

Q2a. Your first version should return a multi-element vector of single character nucleotides. For example `generate_dna(6)` might return “A”, “T”, “T”, “G”, “A”, “C”.

```
generate_dna <- function(len=10) {  
  sample(x=c("A", "C", "G", "T"), size= len, replace= TRUE)  
}
```

```
generate_dna()
```

```
[1] "C" "T" "C" "G" "T" "G" "A" "C" "C" "G"
```

```
generate_dna(len=100)
```

```
[1] "A" "A" "C" "T" "A" "A" "C" "A" "C" "G" "A" "T" "A" "C" "A" "A" "G" "T"  
[19] "C" "G" "C" "T" "T" "G" "C" "C" "T" "T" "T" "G" "C" "G" "T" "G" "T" "T"  
[37] "G" "G" "G" "A" "T" "T" "A" "G" "T" "T" "C" "G" "G" "A" "G" "A" "T" "T"  
[55] "C" "C" "C" "T" "C" "A" "A" "C" "G" "T" "C" "A" "T" "C" "T" "T" "A" "G"  
[73] "G" "C" "T" "T" "T" "G" "A" "T" "T" "A" "T" "T" "C" "C" "T" "C" "C" "C"  
[91] "A" "A" "G" "T" "G" "G" "G" "T" "T" "C"
```

Q2b: Your second version should **optionally** be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a single vector of multiple letters).

```
generate_dna <- function( len=10, single.element=TRUE) {  
  
  ans <- sample(x=c("A", "C", "G", "T"), size=len, replace= TRUE)  
  
  #cat("Hello...")  
  
  if(single.element) {  
    #cat("is it me you are looking for...")  
    ans <- paste( ans, collapse = "")  
  }  
  return(ans)  
}
```

Functions that could be useful here are `paste()`, `if()`, `cat()`, and `return()`

```
generate_dna()
```

```
[1] "GGACACTTAA"
```

```
generate_dna(single.element = FALSE)
```

```
[1] "G" "T" "T" "C" "T" "T" "G" "C" "C" "C"
```

```
paste(c("A", "C", "G"), collapse = "")
```

```
[1] "ACG"
```

Q2C. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length.

```
generate_dna <- function(len=10, single.element=TRUE, fasta=FALSE) {  
  
  ans <- sample(x=c("A", "C", "G", "T"), size=len, replace = TRUE)  
  
  if(fasta) {  
    seq_string <- paste(ans, collapse= "")  
    cat(">len", len, "\n", sep="")  
    cat(seq_string, "\n", sep="")  
    return(invisible(seq_string))  
  }  
  
  if(single.element) {  
    ans <- paste(ans, collapse = "")  
  }  
  return(ans)  
}
```

```
generate_dna(9, fasta=TRUE)
```

```
>len9  
T C G A G T G G G
```

Write a generate_protein() function

Q3. Write a function generate_protein() that returns a random peptide/protein sequence of a length specified by the user. For example generate_protein(6) might return "WQRTAG".

```
generate_protein <- function(length=9) {  
  aa <- c ("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "S", "  
  ans <- sample(x=aa, size=length, replace = TRUE)  
  ans <- paste(ans, collapse = "")  
  return(ans)  
}
```

```
generate_protein(6)
```

```
[1] "VIQVTG"
```

Generate random protein sequences of length 6 to 13

Q4 Adapt and use your generate_protein() function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length).

```
for(l in 6:13) {  
  cat(">id.", l, "\n", sep = "")  
  cat (generate_protein(l), "\n")  
}
```

```
>id.6  
KRNYPT  
>id.7  
RQKITFV  
>id.8  
DMFTAHPR  
>id.9  
HMLYWVECR  
>id.10  
GLYSVFKWEL  
>id.11  
CGLQDEDNLPQ
```

```
>id.12
ASEFCMWLHHAF
>id.13
CCPYSAVVFKMCI
```

BLASTp search against nr — are your peptides “unique in nature”?

Q5: Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database

length	Ide	Cov	Unique
6	100	100	N
7	100	100	N
8	100	88	Y
9	100	89	Y
10	89	90	Y
11	100	82	Y
12	82	92	Y
13	73	100	Y

Q5a. At which sequence length do your randomly generated peptides start to look “unique in nature”(i.e. no 100% coverage + 100% identity hit)? [1 pt]

Sequence Length 8

Q5b. Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. [1 pt]

Short peptides are easy to find in the database because there aren’t that many possible sequences (20^L), so they show up a lot in real proteins. But as the length increases, the number of possible sequences becomes huge compared to the number of known proteins, so longer peptides are less likely to match anything.

connecting your finding to Immunology

Q6.: In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides? [3 pt]

Based on my Q5 results, peptides start to look unique around length ~ 8 , so the minimum length for MHC class II presentation is likely around 9–10 amino acids. Very short peptides are not unique because the sequence space is small (20^L), meaning many identical sequences appear across different proteins. As peptide length increases, the sequence space grows rapidly, making matches less likely. If the immune system presented very short peptides, they would frequently match self-proteins, increasing the risk of autoimmune responses. So, longer peptides improve specificity and help T cells distinguish self from non-self.